Kaggle Hedge Fund Panel Forecasting

Takeaway

This deck rebuilds the project as one professional, chart-first story: panel structure, diagnostics, classical baselines, deep models, and an honest comparison at the end.

Forecasting anonymized hedge-fund series under a weighted multi-series evaluation metric.

The project combines exploratory analysis, stationarity testing, representative-series classical modeling, and adaptive-split recurrent networks. The goal is not just to score models, but to explain when each family succeeds, where the data limits matter, and why some comparisons are methodologically constrained.

TRAIN ROWS	TEST ROWS	FEATURES	HORIZONS
5,337,414	1,447,107	86	1 · 3 · 10 · 25

Project angle

The strongest part of the work is the diagnostic chain: weight concentration, cutoff feasibility, ADF/KPSS, differencing, and lag structure all feed directly into how the classical and deep studies are framed.

Main questions

- What does the hedge-fund panel look like before any modeling?
- Which stationarity and lag diagnostics materially change model choice?
- How do classical and deep models behave under realistic train-window constraints?

Deck promises

- No appendix: all major plot families stay in the main flow.
- Every slide has one explicit message, not just a pasted notebook figure.
- The final comparison keeps the methodological caveat that deep models use adaptive splits.

Agenda

Takeaway The talk moves from the panel and metric, to diagnostics, to model families, and ends with a direct comparison plus project limitations.

1 Problem → 2 Data → 3 EDA → 4 Classical → 5 Deep → 6 Compare → 7 Close

Presentation logic

- Use diagnostics to justify model choices rather than treating EDA as decoration.
- Keep all major plots visible, but anchor each one to a single explanatory message.

What to watch for

- Weight concentration changes how model quality should be read.
- The fixed cutoff is useful but not universally feasible across the panel.
- The final comparison is informative, not perfectly apples-to-apples.

Problem Statement

Takeaway This is a weighted panel-forecasting problem, not a single-series curve-fitting exercise, so series identity, time order, and metric concentration all matter together.

What is forecast

The target y_target must be predicted over future ts_index values for many distinct series defined by code, sub_code, sub_category, and horizon.

Why it is hard

The panel is heterogeneous, heavy-tailed, and only a minority of series cleanly supports the fixed cutoff used for validation in the EDA and classical notebooks.

What counts as success

A useful model improves the weighted skill score on future data without breaking chronology, leaking information, or overclaiming from tiny train segments.

Project structure

- EDA explains the panel before any model comparison.
- Classical models are tested on four representative series.
- Deep models use adaptive splits only when fixed-cutoff training becomes absurdly short.

Dataset Overview

Takeaway The forecasting target lives inside a large anonymized panel: 5.34M train rows, 1.45M test rows, 86 engineered features, and four horizons.

Core schema

Field	Meaning
code	Primary series identifier
sub_code	Secondary series identifier
sub_category	Category grouping
horizon	Forecast horizon
ts_index	Integer time index
y_target	Continuous target value
weight	Per-row competition weight

TRAIN ROWS

5,337,414

TEST ROWS

1,447,107

FEATURE COUNT

86

CODES

23

Panel scale

Train covers ts_index 1–3601 and test continues from 3602–4376. The panel spans 180 sub-codes in train, 47 in test, 5 sub-categories, and 4 horizons.

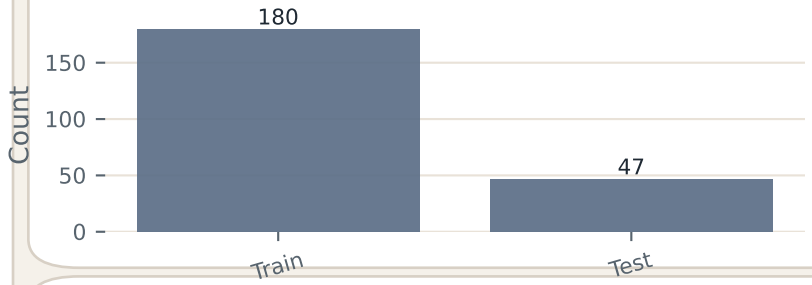
Train vs Test Comparability

Takeaway Train and test share the same schema and category families, but test uses a narrower sub-code universe, so the main risk is coverage shift rather than schema mismatch.

Categorical support check

Column	Train unique	Test unique	Test subset of train
code	23	23	Yes
sub_code	180	47	No (narrower support)
sub_category	5	5	Yes
horizon	4	4	Yes

Sub-code universe



What this means

- Train-only fields are exactly y_target and weight, which is expected.
- Every category level seen in test already exists in train, so there is no true cold-start schema issue.
- The narrower test support still matters because validation and generalization should respect that coverage shift.

Validation Strategy

Takeaway Because test is a later suffix in time, validation must also be chronological; random cross-validation would leak future information.

Timeline view of the forecasting problem

Why the split matters

- The fixed cutoff is the shared boundary used throughout EDA and the classical study.
- Shuffled folds would mix future information into earlier states and overstate performance.

What changes later

- The deep notebook keeps chronology but moves to adaptive per-series splits.
- That change is methodological, not cosmetic, because some chosen series have only seven pre-cutoff points.

Metric Math: Weighted Skill Score

Takeaway All model discussion is anchored to the weighted skill score, because weight concentration makes unweighted RMSE fundamentally misleading.

Primary score used in the project

$$\text{score} = \sqrt{1 - \text{clip}_{0,1}\left(\frac{\sum_i w_i (y_i - \hat{y}_i)^2}{\sum_i w_i y_i^2}\right)}$$

Interpretation

- 1.0 is perfect forecasting.
- 0.0 means no gain beyond the clipped baseline threshold.
- Large weights can dominate the score even if most rows are easy.

Why it changes decisions

The score rewards getting high-weight rows right. That is why the EDA treats weight concentration as a modeling issue, not just a descriptive statistic.

Libraries and Workflow

Takeaway

The project uses a compact Python stack: pandas and pyarrow for data access, matplotlib and seaborn for visuals, statsmodels for classical diagnostics, and torch for sequence models.

EDA notebook → **Classical study** → **Deep study** → **Comparison**

Data and IO

pandas, numpy, pyarrow, pathlib

Used for parquet loading, grouped summaries, window construction, and result tables.

Visualization and diagnostics

matplotlib, seaborn, statsmodels

Used for missingness views, ADF/KPSS, ACF/PACF, STL, and classical forecasting.

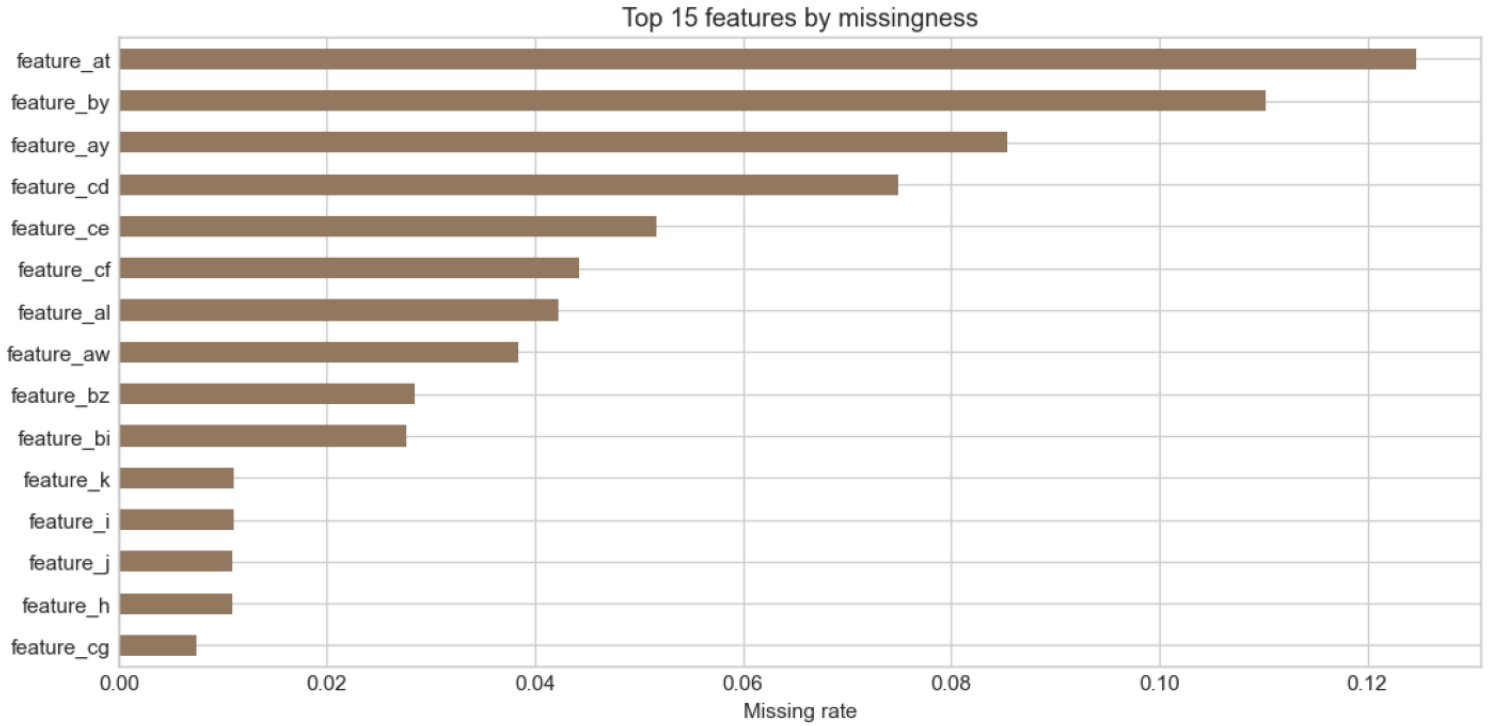
Deep learning

torch, torch.utils.data

Used for one-step window training, GRU/LSTM/RNN models, recursive forecasting, and training-curve tracking.

Missing Values

Takeaway Missingness exists in 48 columns, but the maximum missing rate is only about 12.47%, so the main issue is selective feature reliability rather than catastrophic data loss.



Top missing columns

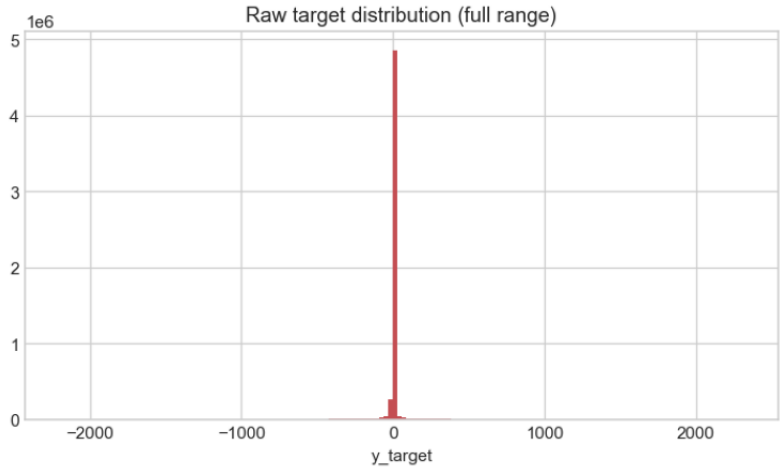
Most missing columns	Missing rate
feature_at	12.47%
feature_by	11.02%
feature_ay	8.54%
feature_cd	7.50%
feature_ce	5.17%

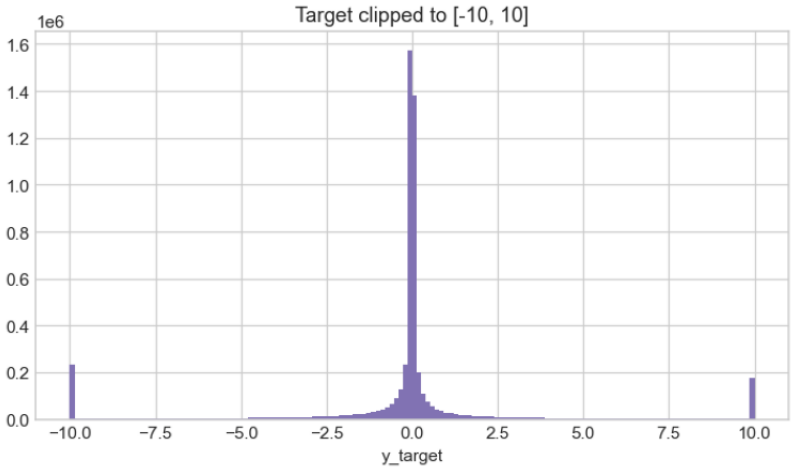
Modeling implication

- The missingness pattern should be monitored when choosing feature sets or imputations.
- The rates are non-trivial, but still low enough that a meaningful feature audit remains possible.

Target Distribution

Takeaway The target is centered near zero but has extremely heavy tails, so outliers can dominate error-based metrics even when most observations are small.





Summary statistics

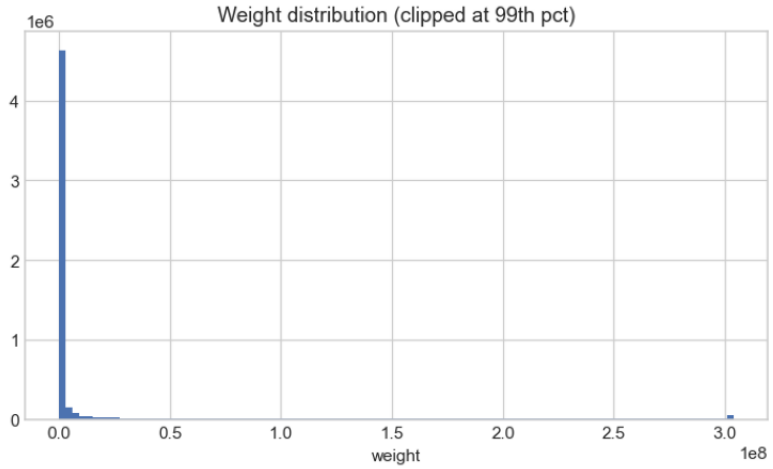
Median: -0.0006
IQR: -0.1291 to 0.0511
1% / 99%: -82.80 / 62.92
Range: -2201.88 to 2314.41

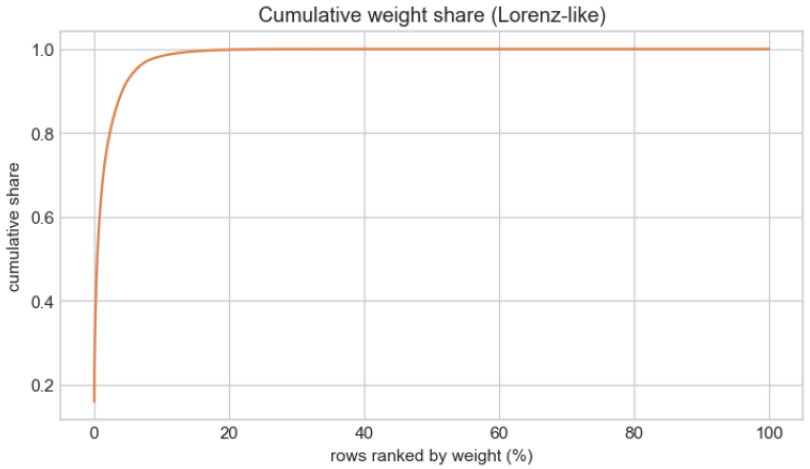
What we learn

- Robustness matters because tail events sit far outside the central mass.
- Simple means can be distorted by rare spikes.
- Visual evaluation still has to be tied back to weighted metrics.

Weight Distribution

Takeaway A tiny fraction of rows carries most of the evaluation weight, so the competition metric and unweighted losses can tell very different stories.





TOP 1%	TOP 5%	TOP 10%
64.19%	**92.58%**	**98.33%**

Scale of concentration

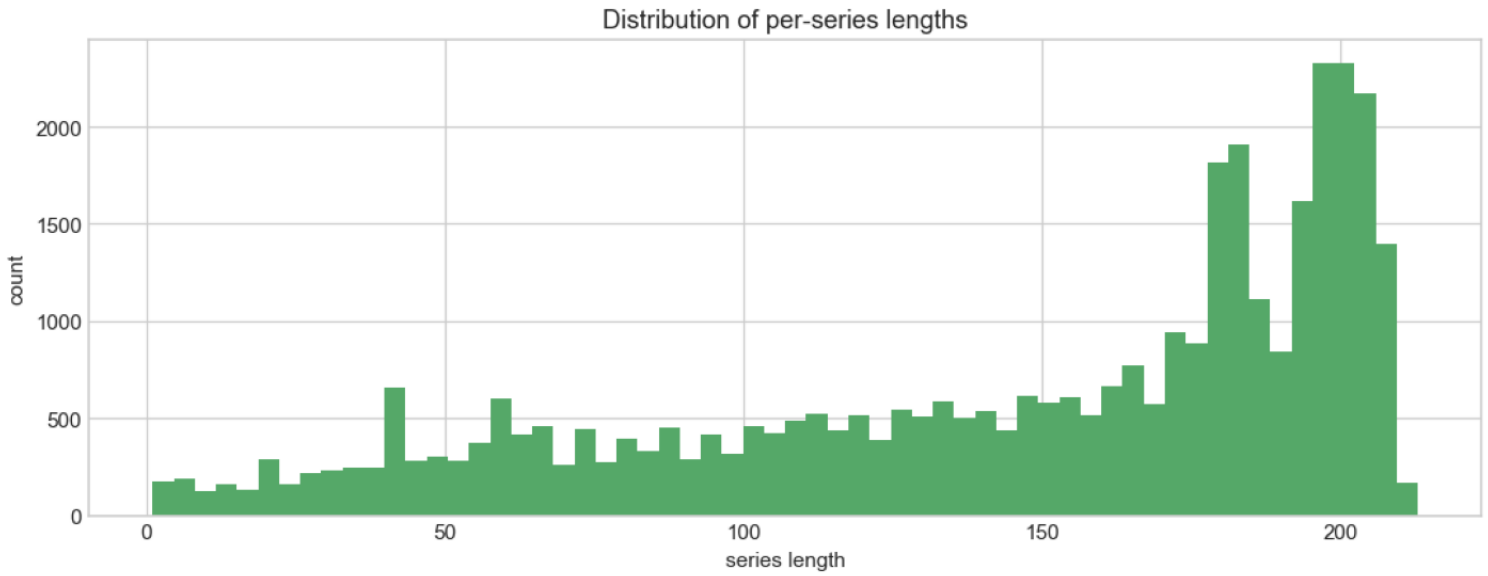
Median weight is 1,699.38, while the maximum weight is 1.391e13. The upper tail is therefore far more extreme than the central tendency suggests.

Implication

- High-weight rows dominate ranking outcomes.
- Any evaluation that ignores weights can recommend the wrong model family.

Series Coverage and Split Feasibility

Takeaway Only about 5% of all series cleanly cross the validation cutoff, so cutoff-based evaluation is informative but not universally feasible across the panel.



TOTAL SERIES

36,923

CROSS CUTOFF

1,930

Key point

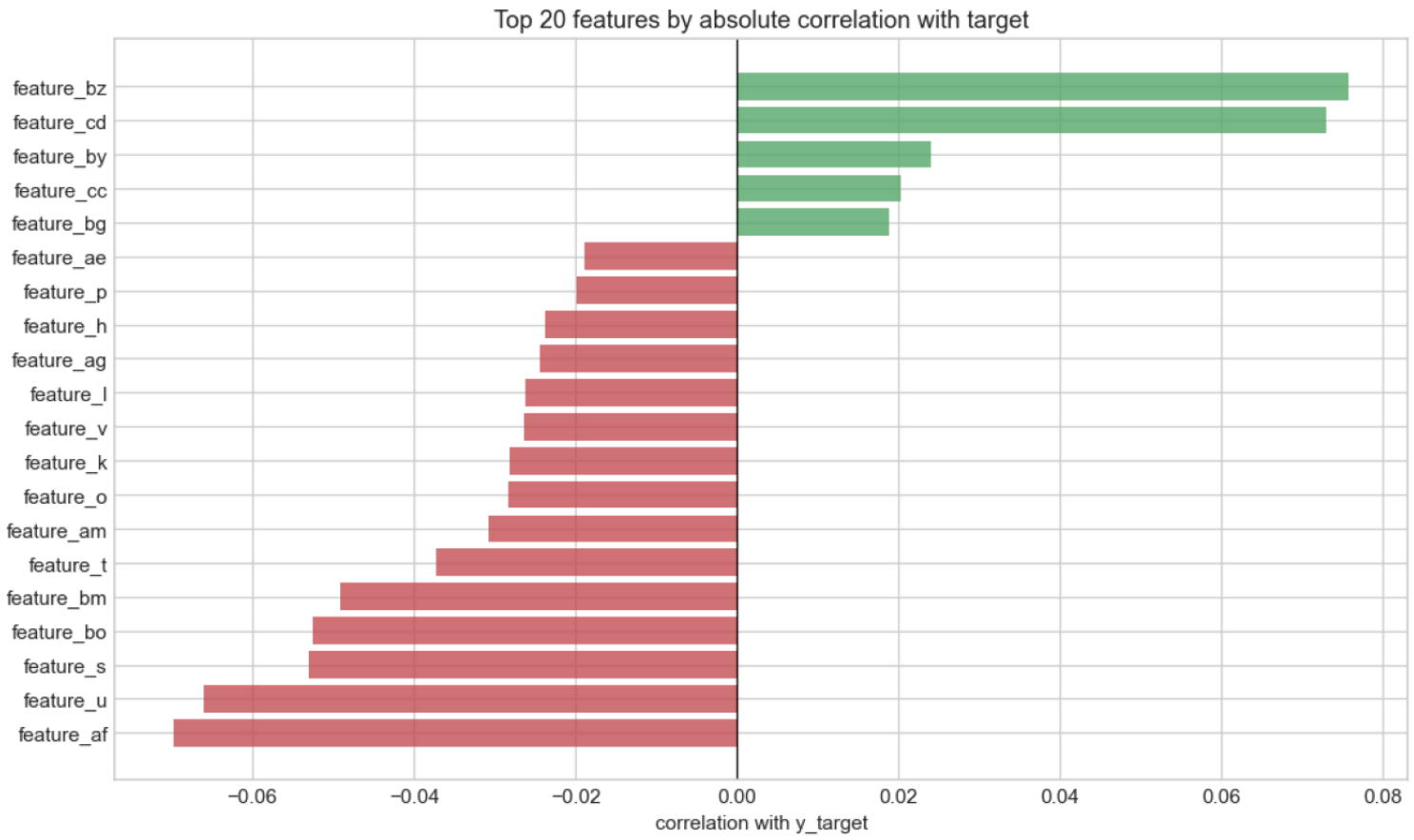
The notebook keeps only eligible series for later diagnostics by enforcing both minimum length and cutoff-crossing requirements.

Why it matters

- Some model families simply do not have enough pre-cutoff data.
- Representative-series studies become a practical compromise.

Feature Audit

Takeaway The 86 anonymized features still show measurable target relationships, but their anonymous naming means we can only reason statistically, not semantically.



Top correlations

Top |corr(y, feature)|	Correlation
feature_bz	0.0758
feature_cd	0.0730
feature_af	-0.0697
feature_u	-0.0660
feature_s	-0.0531
feature_bo	-0.0525
feature_bm	-0.0492
feature_t	-0.0372

Interpretation

- Useful signal exists, but it is modest and spread across many features.
- The audit guides later model choice more than any domain-level interpretation.

Representative-Series Selection Logic

Takeaway The four chosen series were selected to span the panel's length, weight concentration, volatility, and stability extremes rather than to show only easy cases.

Chosen representative series

Reason	Series key	Length	Total weight	Target std
Longest History	X9BZ68VQ / OYJGNSQK / DPPUO5X2 / H1	212	9.06e2	2.6416
Highest Total Weight	SJZP0OVU / OYJGNSQK / NQ58FVQM / H25	156	4.35e10	0.0004
Most Volatile	W4S29LF4 / KL66VIS3 / PHHHVYZI / H25	162	4.56e-1	296.7600
Most Stable	SJZP0OVU / OYJGNSQK / NQ58FVQM / H1	170	3.87e10	0.0001

Why these four

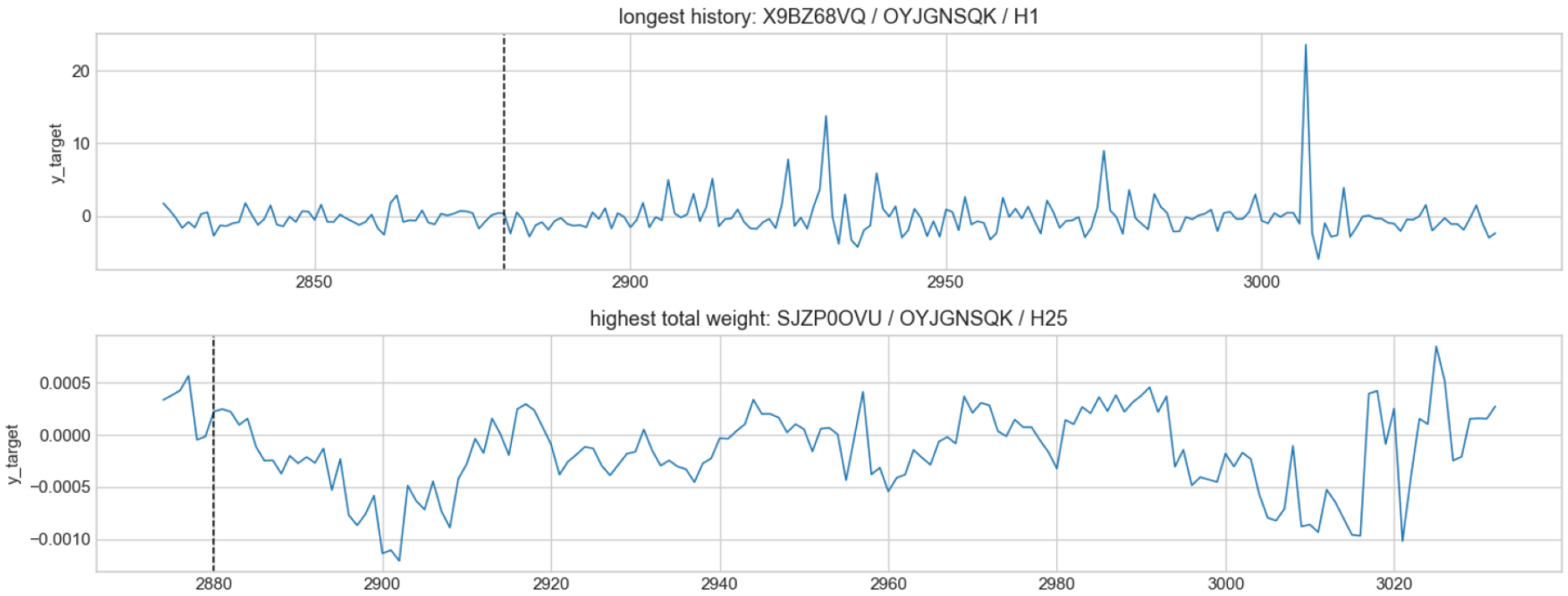
They capture four distinct conditions: plenty of history, huge weight, extreme variability, and near-flat behavior. That makes later visual comparisons interpretable and honest.

What each one stresses

- Longest history tests persistent lag structure.
- Highest total weight tests metric-sensitive behavior.
- Most volatile tests spike handling.
- Most stable tests tiny-scale numerical accuracy.

Representative-Series Raw Plots I

Takeaway The first two representative series already show the panel contrast: one has meaningful lag structure, while the high-weight series is tiny in scale and much harder to interpret visually.

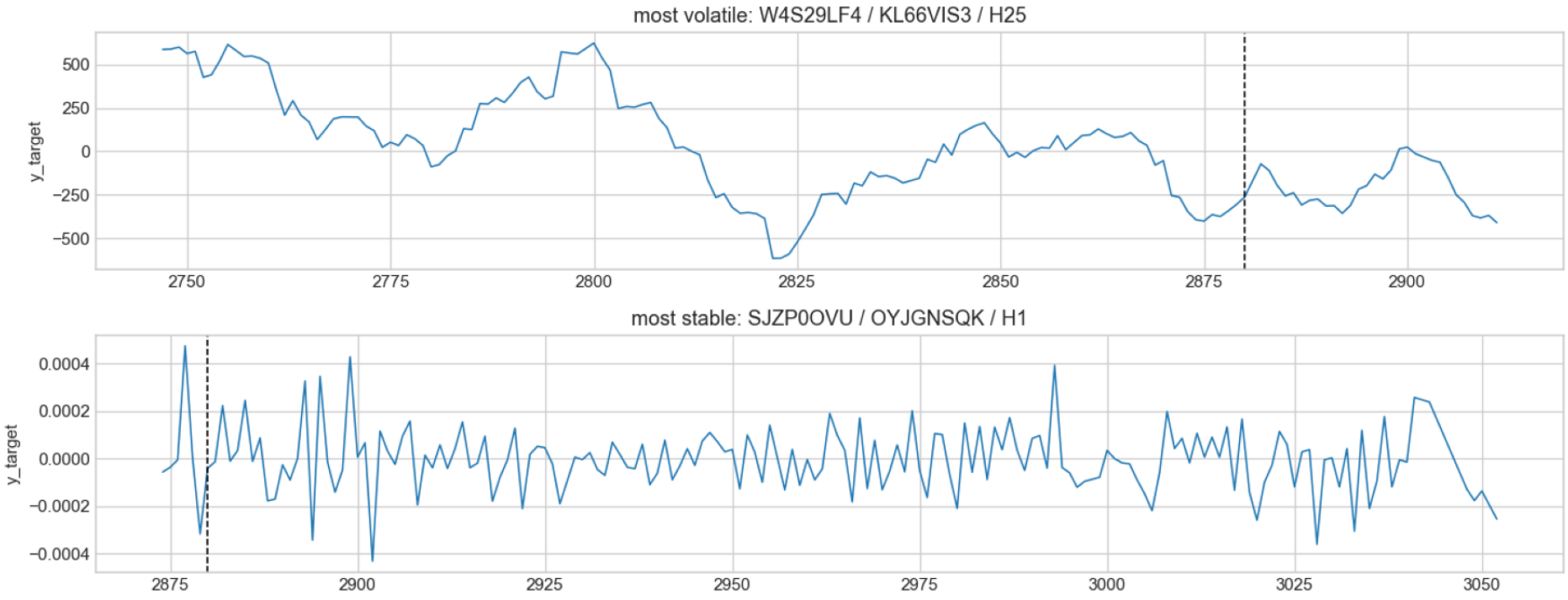


Visual message

These two series show why train-window length and target scale matter before any model is fit.

Representative-Series Raw Plots II

Takeaway The second pair completes the contrast: the volatile series has obvious structure and swings, while the most-stable series is close to a numerical-precision problem.



Visual message

This is why a single model story is hard to defend. The panel contains both visible structure and series that are almost numerically flat.

ADF Test

Takeaway ADF is the first stationarity gate: if many raw series fail it, classical models in levels should not be trusted without transformation.

Null and alternative

H_0: the series has a unit root (non-stationary)
H_1: the series is stationary

Reject H_0 when p-value < 0.05.

Why it matters here

ARIMA-style models assume a stable mean and autocorrelation structure after any differencing. Running ADF first avoids blindly treating trending series as stationary.

Notebook setup

One ADF test is run per eligible series, with all four series keys retained on each result row. The notebook samples 200 eligible series to keep panel-level conclusions readable and practical.

Decision rule used later

- If many raw series fail ADF, first differencing becomes the default repair step.
- If most pass, low-differencing classical models remain viable on the eligible subset.

ADF Results

Takeaway The sampled ADF results show mixed behavior rather than universal stationarity: about 69% pass at 5%, leaving a meaningful minority that still needs repair.

SAMPLED

200

STATIONARY

69%

Reading the figure

- The p-value histogram has a strong left-of-threshold mass, but not a complete collapse toward zero.
- The by-horizon and by-code views show that stationarity is not uniform across the panel.
- That heterogeneity justifies a cautious, diagnostics-driven classical setup.

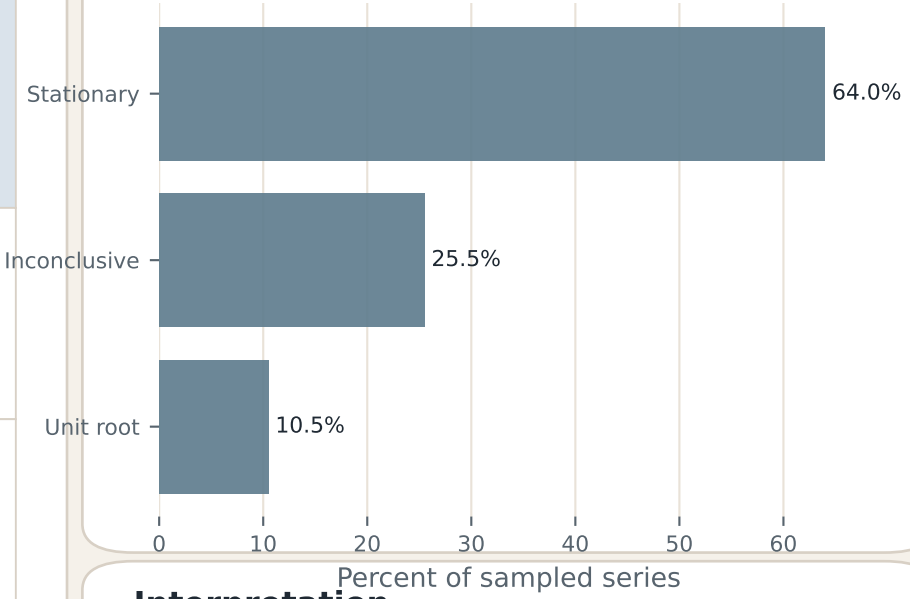
KPSS Test and Joint Verdict

Takeaway KPSS complements ADF by flipping the null hypothesis; together the tests show that most sampled series are usable after mild transformation, but ambiguity remains.

ADF + KPSS combined verdict

Verdict	Share	Interpretation
Stationary	64.0%	ADF rejects unit root and KPSS does not reject stationarity.
Inconclusive	25.5%	Conflicting evidence; often trend-stationary or noisy.
Unit root	10.5%	Fails ADF and rejects KPSS, so differencing is needed.

Verdict shares



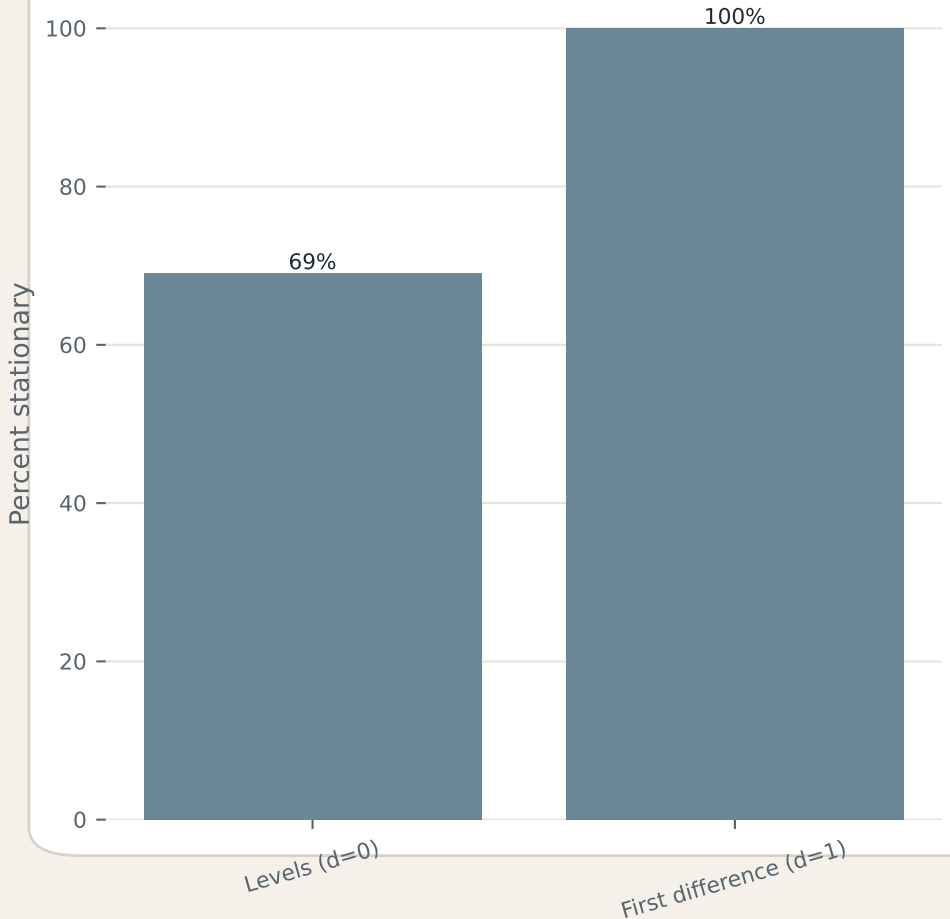
Interpretation

The stationary share dominates, but the inconclusive bucket is large enough that one-size-fits-all claims about raw levels would be unsafe.

First-Differencing Analysis

Takeaway First differencing is a strong repair step in this sample: the stationary share rises from 69% in levels to 100% after differencing.

Before vs after differencing



Mathematical idea

First difference: $\Delta y_t = y_t - y_{t-1}$

Removes slow-moving level shifts.
Stabilizes mean and variance for many series.

Why it matters later

- ARIMA variants with d=1 become well motivated.
- Any level-based model on raw data should now be interpreted carefully when the series still trends.

Rolling Mean / Variance on Raw Series

Takeaway The rolling statistics on raw levels visually confirm what the tests suggest: several chosen series do not maintain a stable mean or variance over time.

Pointwise reading

- A drifting rolling mean indicates non-stationarity in level.
- A widening or narrowing rolling standard deviation indicates non-stationarity in scale.
- The volatile series is especially useful here because its instability is visible even without a formal test.

Rolling Mean / Variance After First Differencing

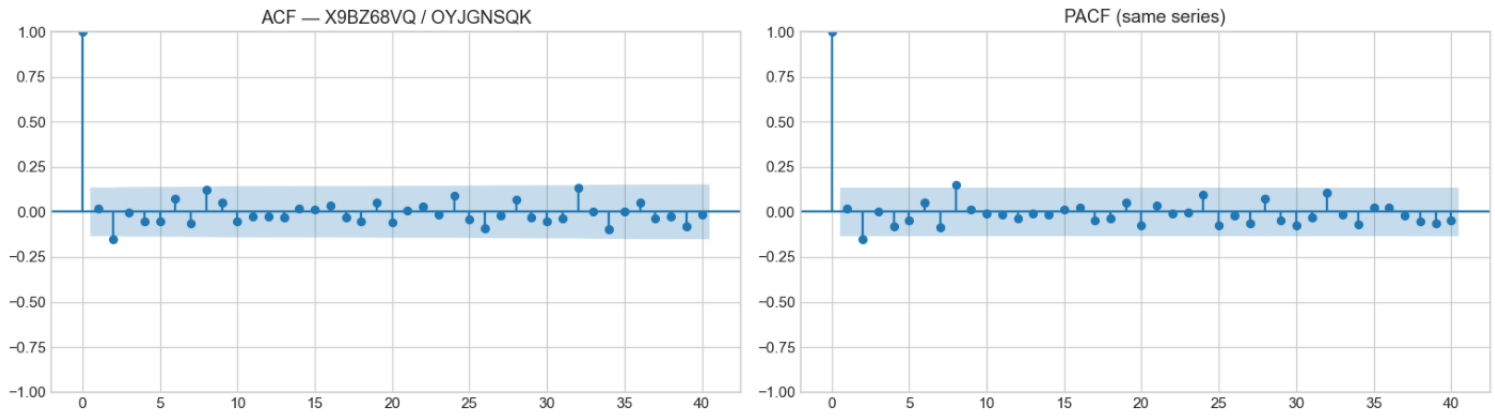
Takeaway After differencing, the rolling mean and variance flatten noticeably, which visually confirms the statistical improvement seen in the ADF results.

Pointwise reading

- The mean now oscillates closer to zero instead of drifting.
- Variance becomes more stable across time windows.
- This is the visual behavior we want before fitting differenced classical models.

ACF / PACF for an Anchor Series

Takeaway The anchor-series correlograms expose which lags matter directly and which reflect shorter-lag mediation, giving concrete hints for AR and MA order selection.

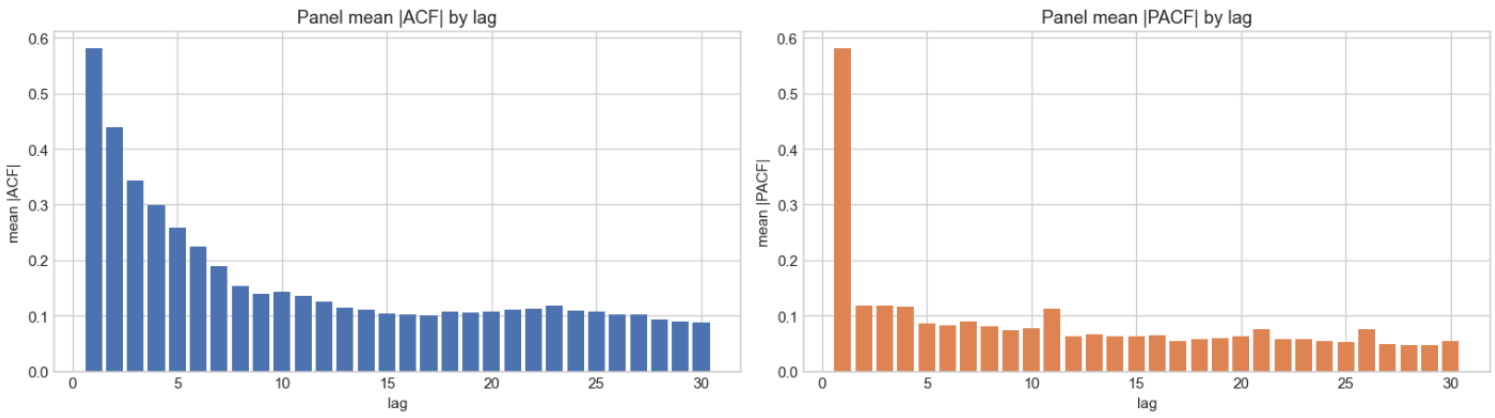


How to read it

- ACF shows correlation with lagged copies of the series.
- PACF isolates the direct contribution of each lag after controlling for shorter lags.
- A sharp PACF cutoff hints at AR order; a sharp ACF cutoff hints at MA order.

Panel-Level ACF / PACF

Takeaway Aggregating correlograms across sampled series shows what lag structure is typical for the panel, not just for one convenient example.

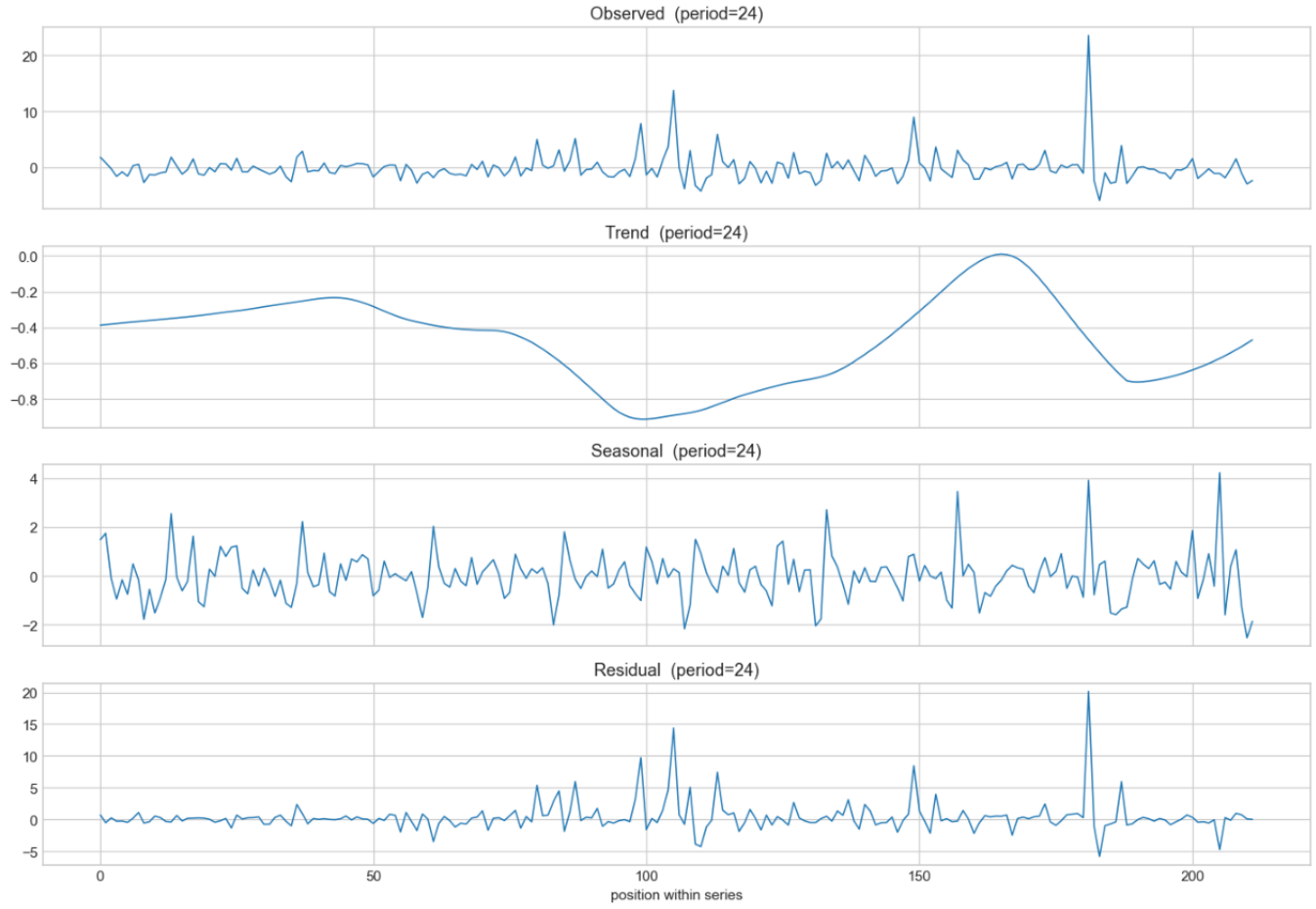


Modeling message

- The panel-wide view prevents overfitting model orders to one series.
- Persistent early lags support low-order autoregressive structure.
- The lack of clean seasonal bumps keeps the focus on short-memory dynamics rather than heavy seasonality.

STL Decomposition

Takeaway STL makes the time-series structure more interpretable by separating observed behavior into trend, seasonal, and residual components.

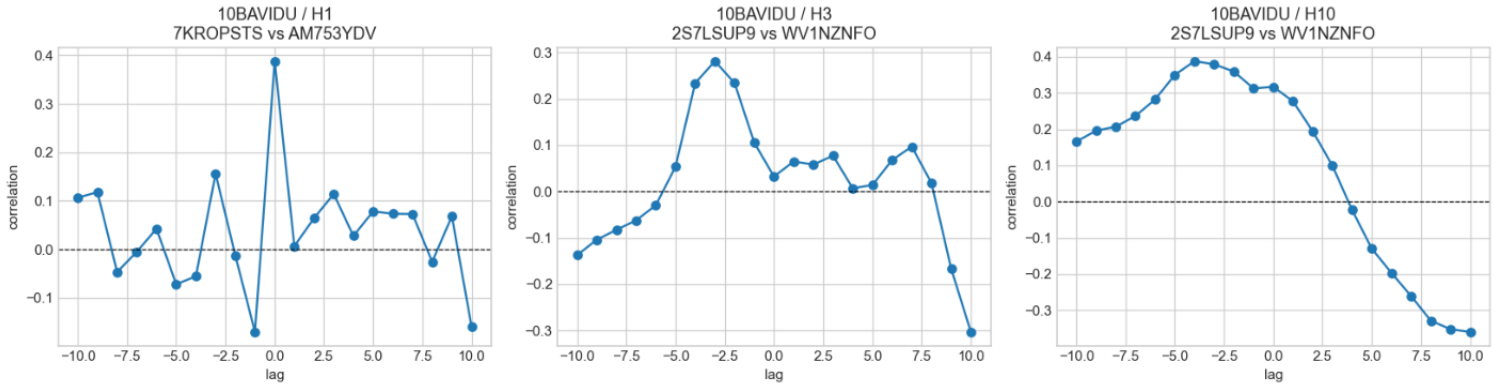


Interpretation

- A sloped trend component matches the non-stationarity seen in earlier tests.
- A weak seasonal amplitude argues against building the whole solution around seasonality.
- Residual structure shows whether the decomposition captured enough of the series behavior.

Cross-Series Relationships

Takeaway The panel is not a bag of isolated series: shared behavior within codes suggests that pooled or global models can potentially borrow signal across related sub-series.



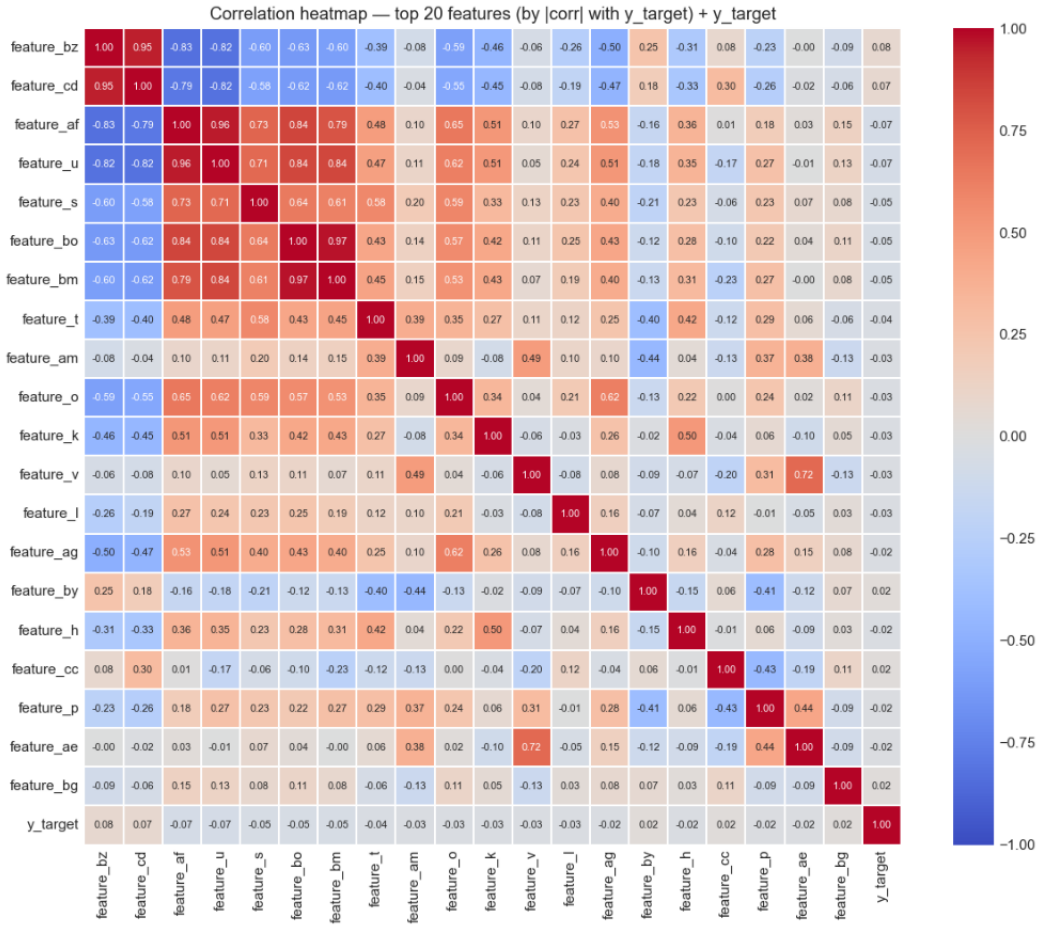
Why this matters

- If related sub-series co-move, a global model can use pooled structure more effectively than isolated per-series fitting.
- This is especially relevant when individual series are short or noisy.
- It explains why the project later compares local classical models against pooled deep-learning behavior.

Top-Feature Correlation Heatmap

Takeaway

The top features are not independent; several high-ranking variables are correlated with one another, so importance should be read with collinearity in mind.



Reading the heatmap

- The first row and column anchor feature-to-target relationships.
- Strong feature-feature blocks suggest redundancy rather than independent signal sources.
- That limits naive marginal interpretation and supports careful model choice.

Leakage and Split Sanity

Takeaway The raw schema does not expose an obvious leakage path: the only train-only fields are the target and the weight, which is exactly what we expect.

Row counts by partition

Partition	Rows
train_pre_cutoff	4,121,749
train_post_cutoff	1,215,665
test	1,447,107

Safety checks performed

The notebook verifies train-only and test-only columns, confirms that y_target and weight are absent from test, and separates pre-cutoff from later rows to preserve temporal causality.

Practical conclusion

- No schema-level leakage is obvious.
- The bigger challenge is respecting time order and cutoff feasibility, not hidden future columns.

EDA Synthesis

Takeaway The EDA narrows the modeling space: respect time order, evaluate with weights, difference when needed, and be honest about series-length constraints.

Data split

Use chronological validation only; test is a true temporal suffix.

Metric

Weighted skill score is non-negotiable because the weight distribution is extremely concentrated.

Stationarity

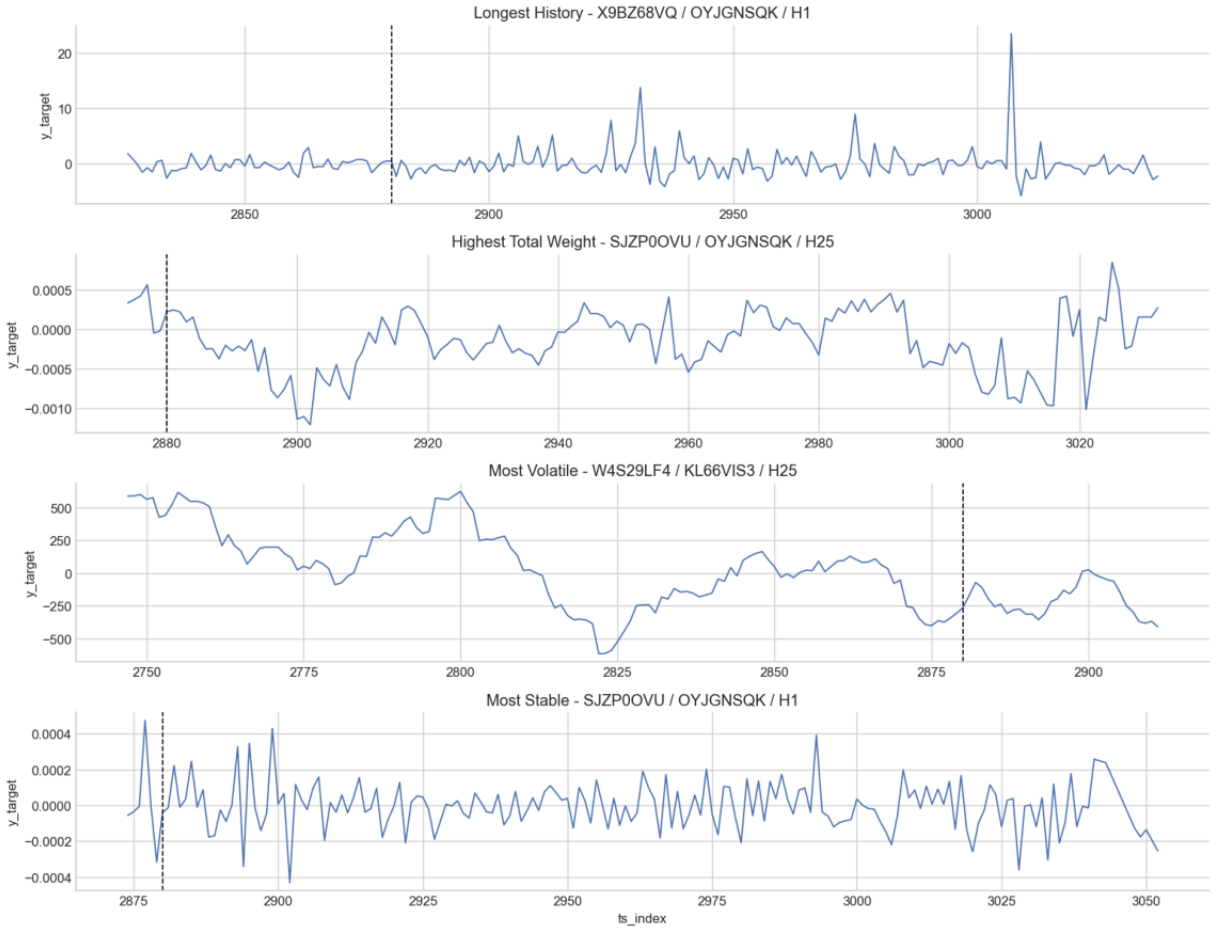
Differencing repairs most non-stationarity; classical level models need caution.

Modeling consequences

- Short-memory dynamics matter more than strong seasonality.
- Cross-series dependence suggests value in global models.
- Many series are too short for a strict cutoff study, which motivates adaptive deep splits.

Classical Study Setup

Takeaway The classical notebook stays intentionally small and interpretable by focusing on four representative series rather than pretending every series supports the same level-based experiment.



Why this design

- The four chosen series already span the practical edge cases discovered in EDA.
- Showing raw history first makes later forecast behavior easier to interpret.
- A small honest study is better than hiding weak train windows inside aggregate tables.

Classical Model Lineup

Takeaway The lineup mixes smoothing, autoregressive, moving-average, and differenced ARIMA families so we can see which level of structure each representative series supports.

Model families included

Family	Models	Min trai	Intuition
Smoothin	Rolling Mean (w=10)	2	Constant average of latest window
Smoothin	SES	3	Single smoothed level, flat multi-step forecast
Smoothin	Holt	5	Level + trend extrapolation
AR	AR(1), AR(2), AR(3)	20	Lagged target dependence only
MA	MA(1), MA(2), MA(3)	20	Uses serially correlated shocks
ARMA	ARMA(1,1) to ARMA(3,3)	20	AR and MA combined in levels
ARIMA	ARIMA(0,1,1), ARIMA(1,1,0), ARIMA(1,1,1)	20	Difference first, then fit ARMA

Reading the lineup

- Smoothing models are cheap and robust on tiny histories.
- AR, MA, and ARMA need at least 20 training points, so some series are skipped by design.
- The ARIMA block tests whether first differencing helps in the volatile case.

Classical Model Math

Takeaway The family formulas matter because the later forecast plots make sense only if we remember which models naturally flatten, trend, or react to shocks.

Rolling / SES / Holt

Rolling mean: $\hat{y}_{t+1} = \frac{1}{w} \sum_{i=0}^{w-1} y_{t-i}$

SES: $\ell_t = \alpha y_t + (1-\alpha)\ell_{t-1}$

Holt adds a trend term on top of the level.

AR and MA

AR(p): $y_t = c + \sum_{i=1}^{p} \phi_i y_{t-i} + \varepsilon_t$

MA(q): $y_t = \mu + \varepsilon_t + \sum_{i=1}^{q} \theta_i \varepsilon_{t-i}$

ARMA / ARIMA

ARMA combines lagged values and shocks.

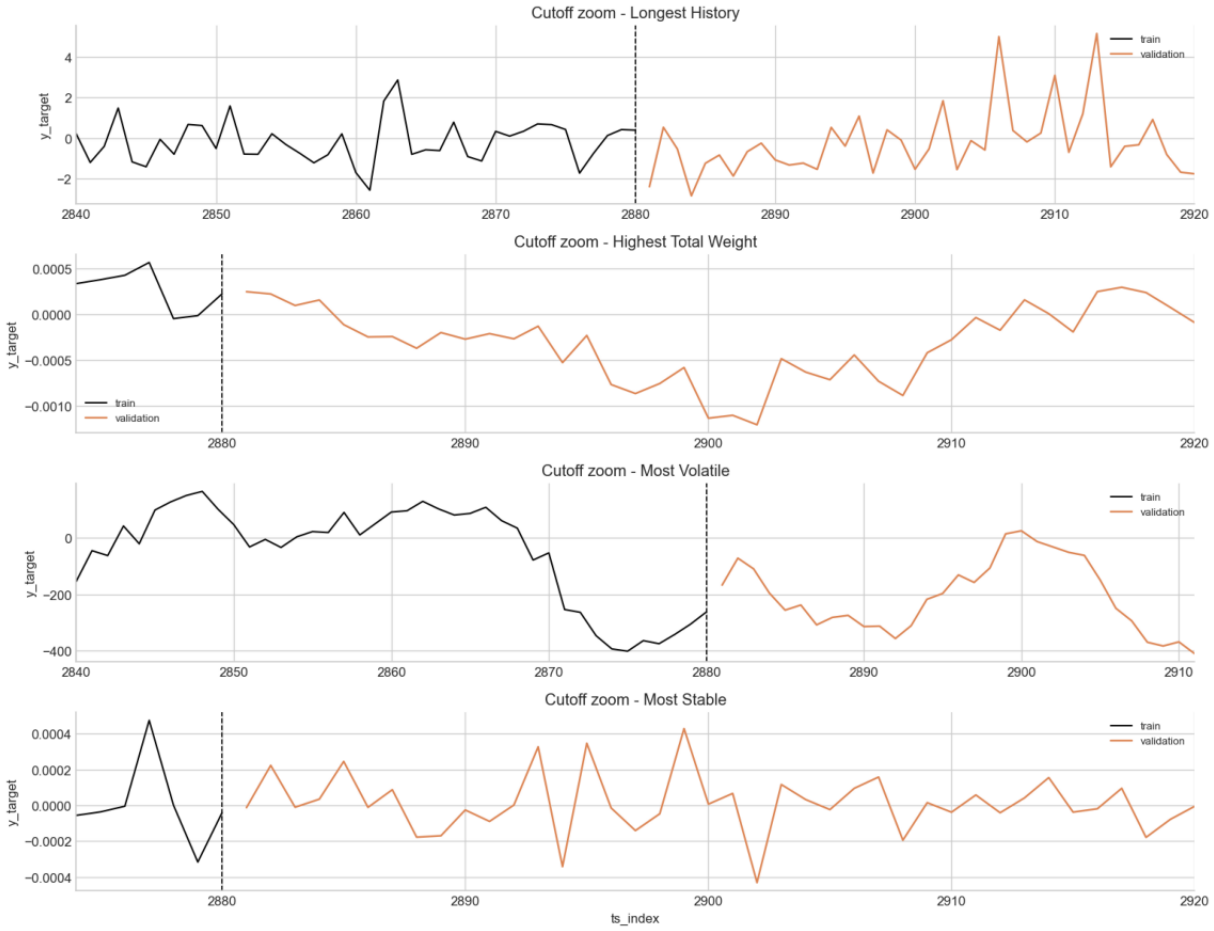
ARIMA(d=1) differences first, then fits ARMA on the transformed series.

Visual consequence in later plots

- Rolling mean and SES often become almost flat multi-step forecasts.
- Holt can extrapolate a straight trend line even when the true series bends.
- AR, MA, ARMA, and ARIMA still look simple when the train segment is short.

Classical Evaluation Protocol

Takeaway The cutoff zoom shows why evaluation is hard here: two of the four representative series offer only seven pre-cutoff training points, which immediately constrains model complexity.



Protocol summary

- Fit each model on ts_index <= 2880 for the chosen series.
- Forecast the entire post-cutoff segment in one multi-step pass.
- Report skill, RMSE, MAE, and MASE, and explicitly mark skipped fits.
- Keep skipped models in the study because train-length failure is itself a result.

Classical Per-Series Leaderboard

Takeaway The best classical model depends heavily on series type; the volatile series is the only one where a differenced ARIMA variant achieves clearly positive skill.

Best classical model by representative series

Series	Best model	Family	Skill	RMSE	MAE	Train len	Val len
Highest Total Weight	SES	Smoothing	0.0000	0.0006	0.0005	7	149
Longest History	ARMA(1,3)	ARMA	0.0000	3.0400	1.6685	55	157
Most Stable	SES	Smoothing	0.0000	0.0001	0.0001	7	163
Most Volatile	ARIMA(1,1,1)	ARIMA	0.8412	129.8104	108.8193	131	31

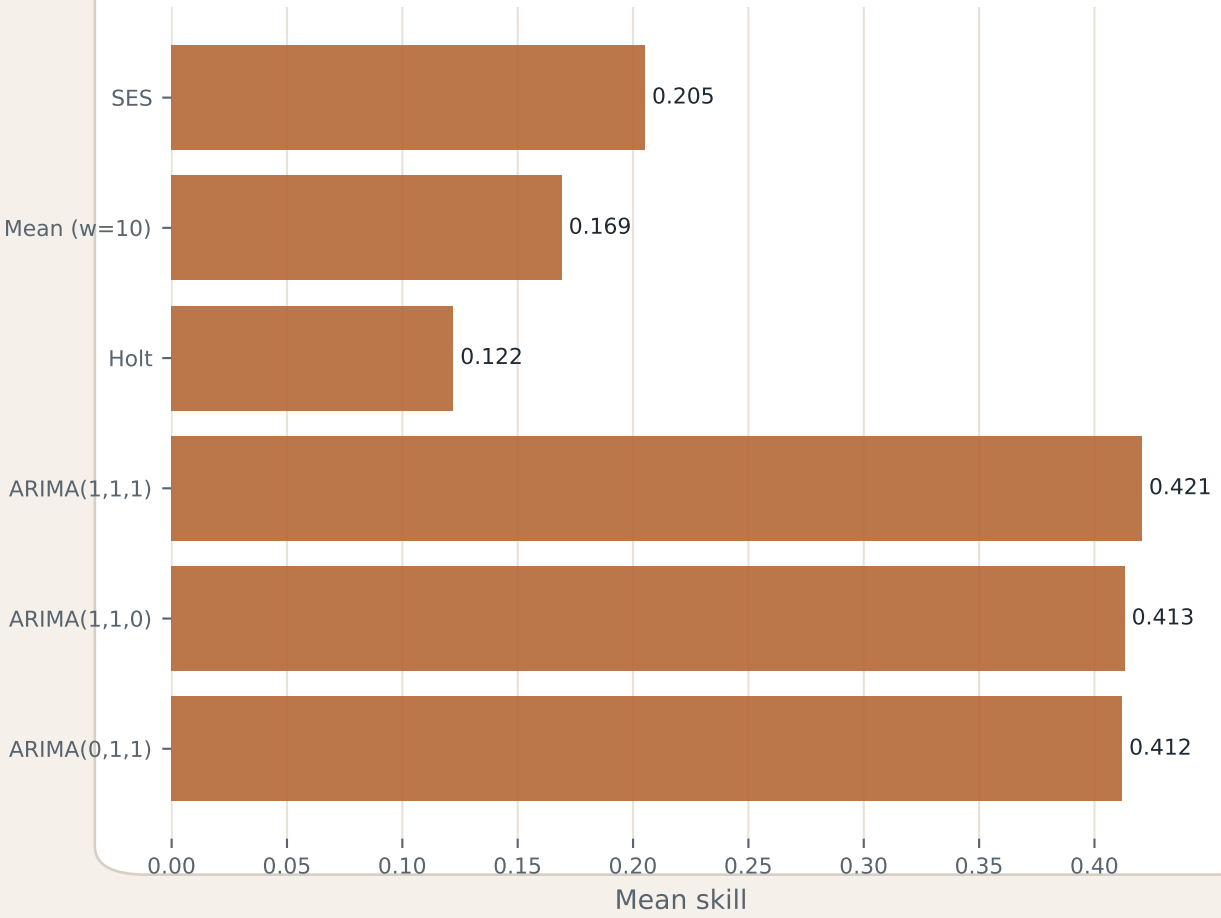
Reading the table

- SES wins the weight-heavy and stable series because there is almost no shape to learn from seven points.
- ARMA(1,3) only edges out others on the longest-history series, and even there skill stays at zero.
- ARIMA(1,1,1) is the clear winner on the volatile series, matching the differencing story from EDA.

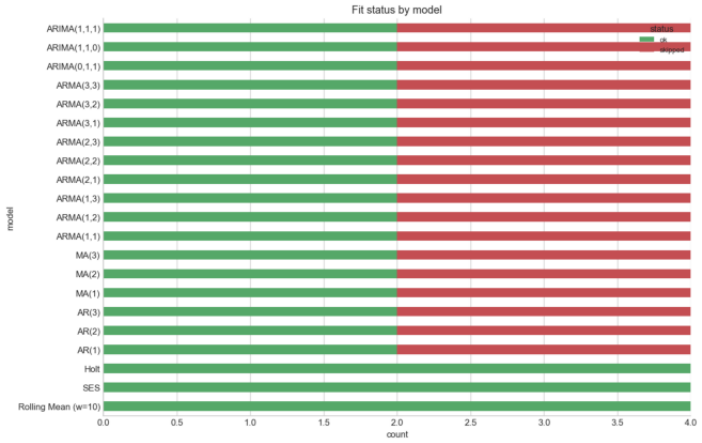
Classical Overall Model Comparison

Takeaway Overall averages should be read cautiously, but they still show two themes: smoothing models always fit, and differenced ARIMA is the only complex family with strong upside on the volatile case.

Top models by mean skill



Fit status by model



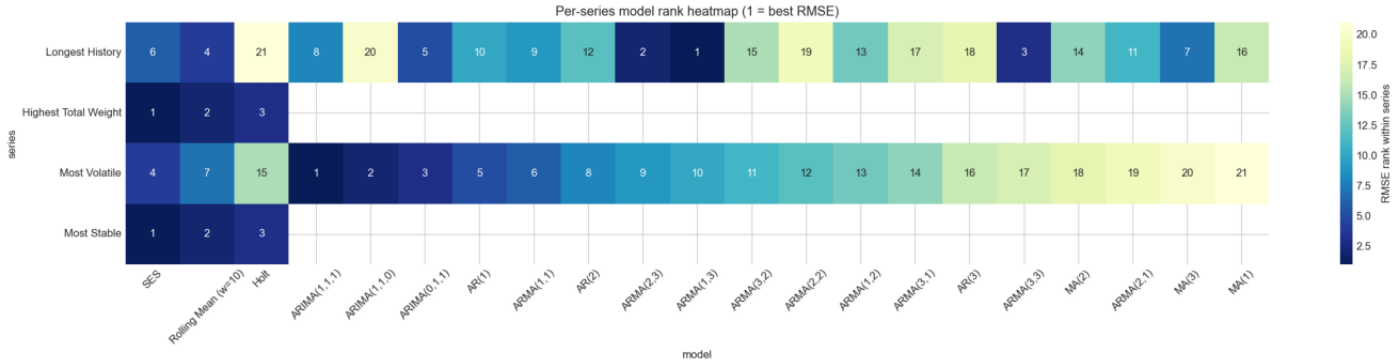
What the status chart adds

Smoothing models are fully feasible on all four series, while many AR, MA, ARMA, and ARIMA variants are skipped on the short-history cases because train_len < 20.

Order Sensitivity and Parameter Patterns

Takeaway Order selection matters, but the visual summaries confirm that tuning only helps when the train window is long enough to support it.

Per-series model-rank heatmap

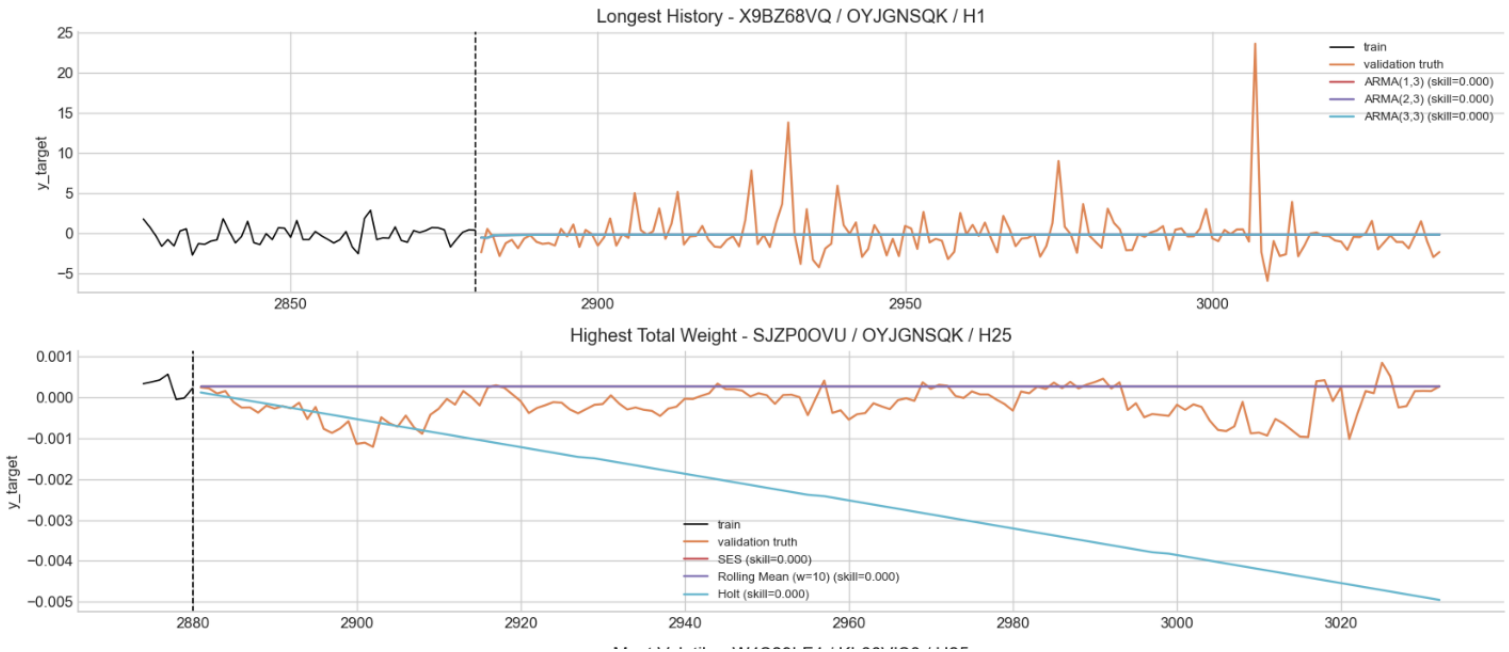


AR / MA order plots and ARMA heatmap

Classical Forecast Overlays I

Takeaway The top-half forecast overlays show two extremes: one moderately dynamic series where many models collapse toward similar shapes, and one nearly flat high-weight series where level models dominate by default.

Longest History and Highest Total Weight



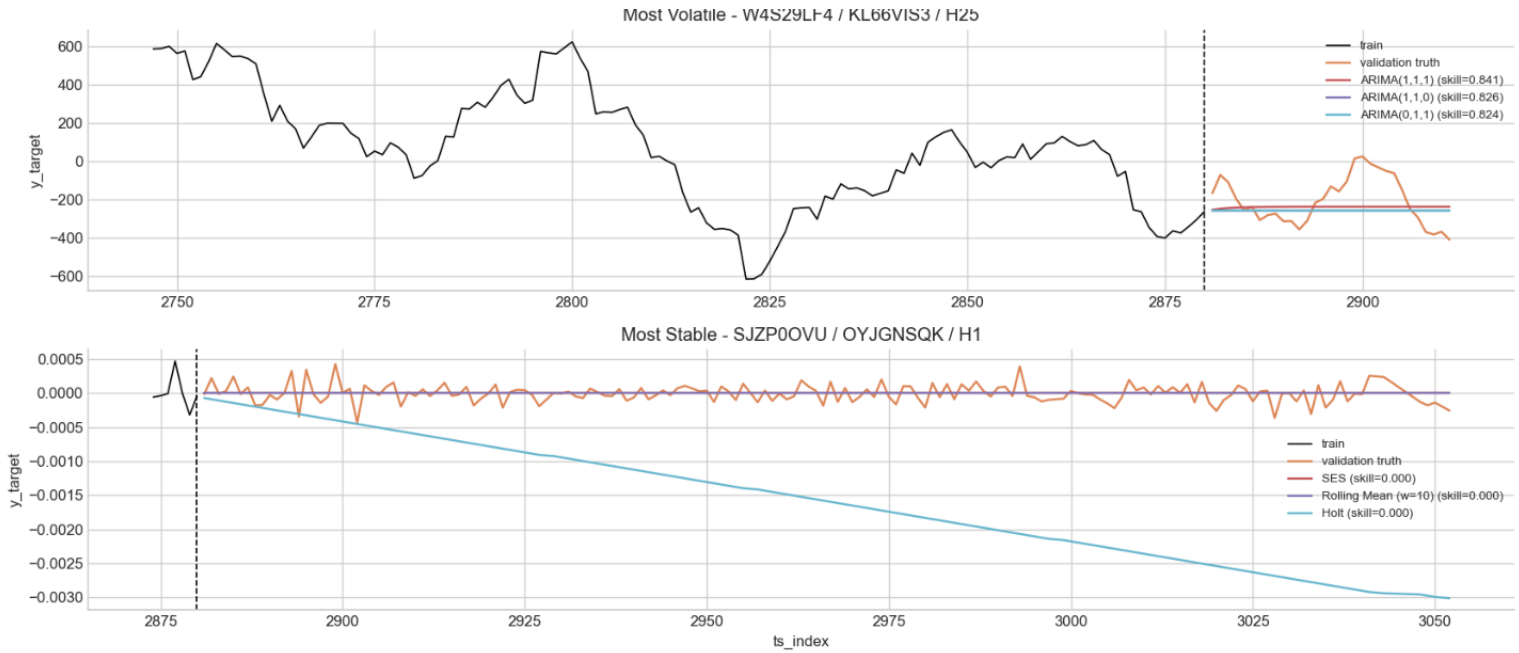
Interpretation

The first pair shows why train-window length and target scale matter. More history helps, but even the longest-history case does not automatically become easy.

Classical Forecast Overlays II

Takeaway The bottom-half overlays make the key contrast obvious: the volatile series benefits from differencing, while the stable series offers so little movement that sophisticated models have almost nothing to exploit.

Most Volatile and Most Stable



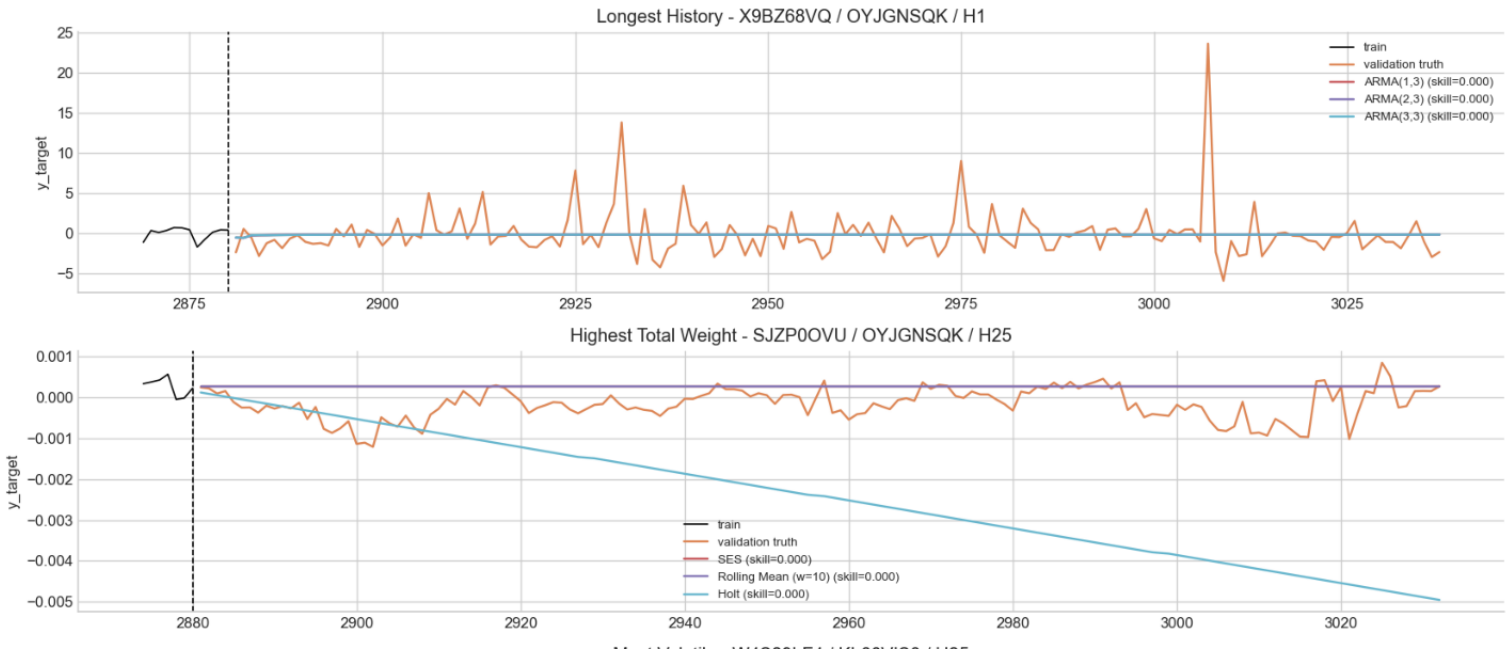
Interpretation

This pair is where ARIMA earns its keep. The volatile series has real structure to capture, while the stable series mostly becomes a precision problem.

Classical Validation Zoom I

Takeaway The validation zoom removes long-history context and shows the actual forecasting challenge around the cutoff, where short-train problems become most visible.

Cutoff zoom: Longest History and Highest Total Weight



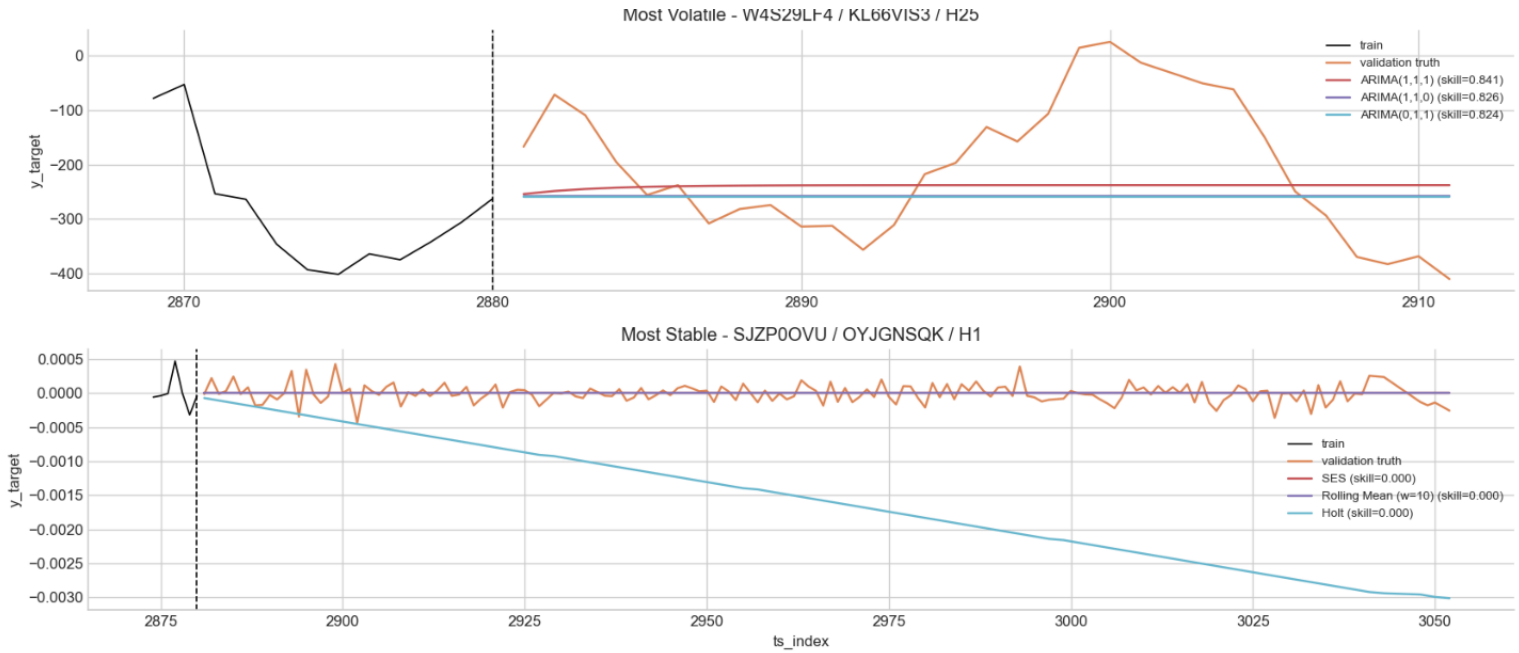
Interpretation

The zoom emphasizes what the models actually saw and predicted near the evaluation window, not just how they look over the whole history.

Classical Validation Zoom II

Takeaway The zoomed volatile and stable series show the real behavior: ARIMA tracks the volatile case better, while the stable case remains almost a numerical-precision exercise.

Cutoff zoom: Most Volatile and Most Stable

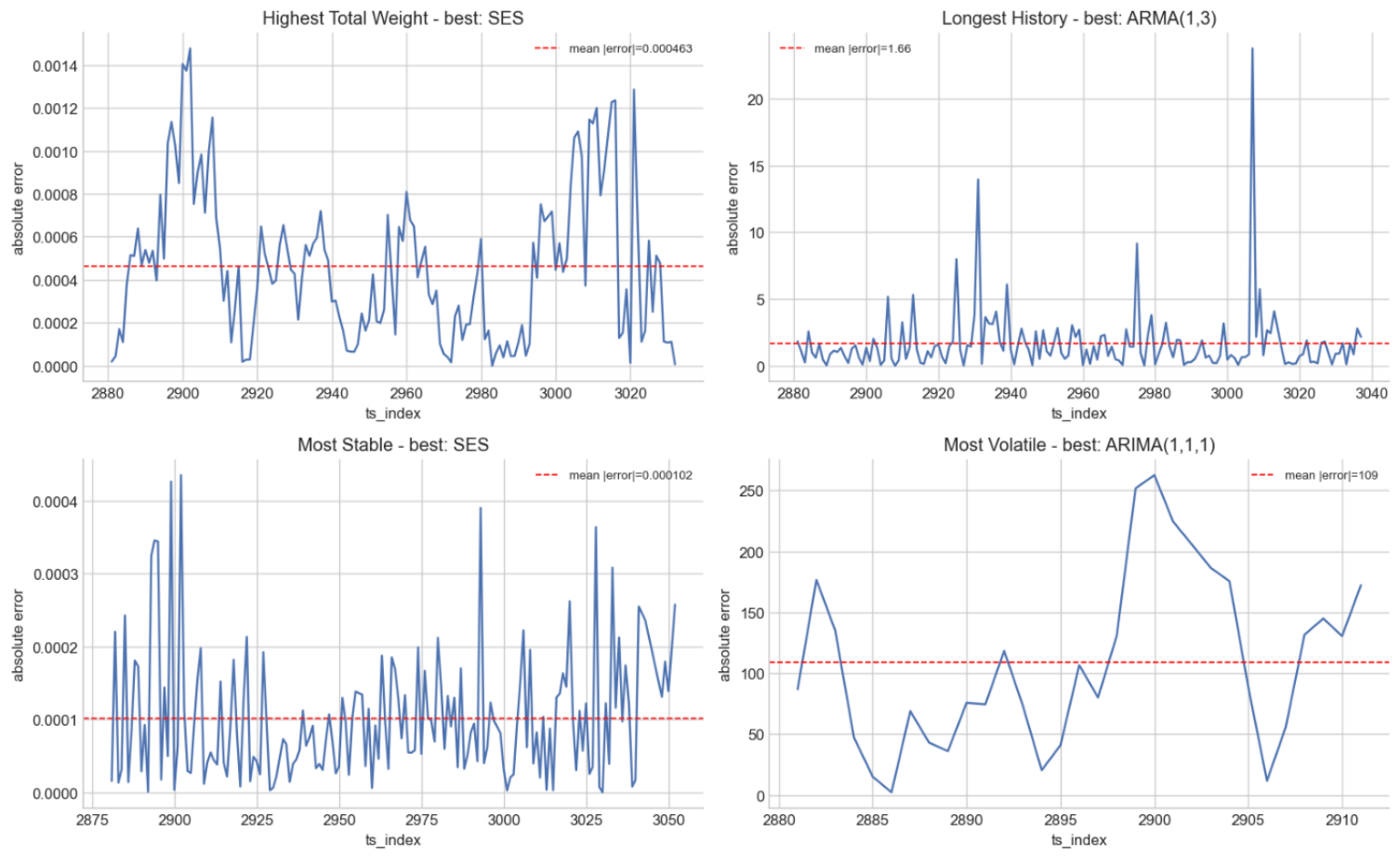


Interpretation

The stable case can make many models look similar. The volatile case is where model family choice has a visibly meaningful effect.

Classical Best-Model Error Plots

Takeaway Looking at error over time is more informative than a single scalar score, because it shows whether a model is consistently a bit wrong or badly wrong at a few decisive spikes.



Pointwise reading

- The volatile series has large concentrated error swings, which is why model choice matters there.
- The stable and high-weight series are numerically tiny, so even visually small deviations can matter under weighting.
- The longest-history series remains challenging despite having more pre-cutoff points.

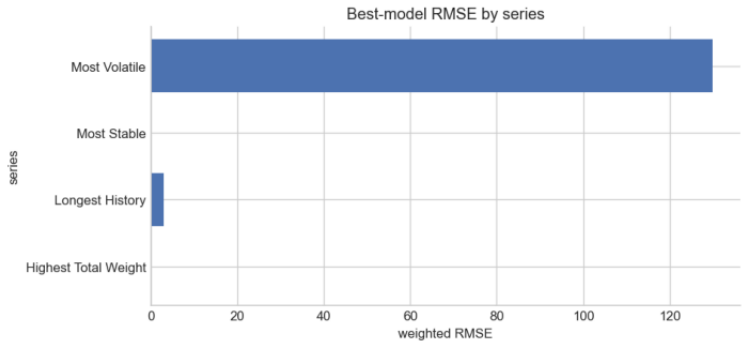
Classical Issues Faced

Takeaway The classical study is useful precisely because it exposes its limits: tiny train windows flatten forecasts, reduce feasible models, and cap the value of order tuning on some series.

Winning model and train/validation lengths

Series	Best model	Family	Skill	RMSE	MAE	Train len	Val len
Highest Total Weight	SES	Smoothing	0.0000	0.0006	0.0005	7	149
Longest History	ARMA(1,3)	ARMA	0.0000	3.0400	1.6685	55	157
Most Stable	SES	Smoothing	0.0000	0.0001	0.0001	7	163
Most Volatile	ARIMA(1,1,1)	ARIMA	0.8412	129.8104	108.8193	131	31

Best-model RMSE by series



Main issues

- Two representative series have only 7 pre-cutoff points, forcing most complex models to skip.
- Flat or straight forecasts are often a genuine model consequence, not a plotting bug.
- Fit feasibility itself becomes part of the result, not just an implementation nuisance.

Classical Takeaways

Takeaway There is no universal classical winner: smoothing is safest on short or near-flat series, while differenced ARIMA is the only family with strong upside on the volatile case.

Safe models

SES and rolling averages remain useful because they still behave sensibly when the training window is tiny.

Where complexity helps

ARIMA(1,1,1) stands out on the volatile series because differencing actually addresses the underlying non-stationarity.

What not to overclaim

More parameters do not rescue the high-weight and stable series when there is almost no pre-cutoff structure to estimate.

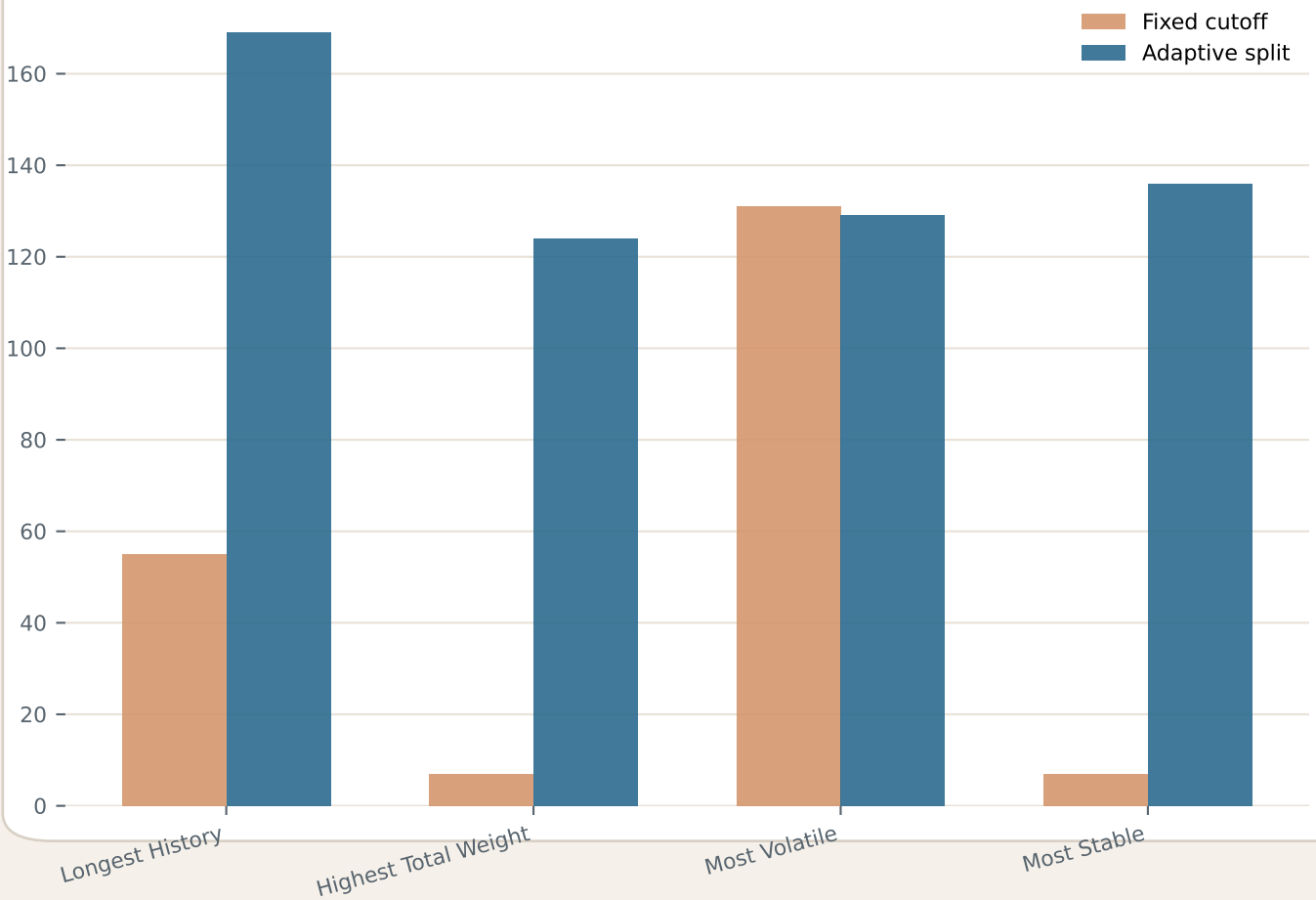
Transition to deep models

- The cutoff problem motivates adaptive splits for recurrent models.
- That change aims to make training feasible without abandoning chronology.

Deep-Study Motivation

Takeaway The deep notebook changes the validation rule for a practical reason: the fixed cutoff leaves two chosen series with only seven training points, which is not enough for sequence modeling.

Train length: fixed cutoff vs adaptive split



Why the change is justified

- Target-only recurrent models need enough sequential context to learn more than a constant baseline.
- Adaptive chronological splits keep train/validation order intact while avoiding absurd 7-point training windows.
- The final comparison therefore has to acknowledge a methodological mismatch.

Deep Setup and Libraries

Takeaway The workflow stays compact: pandas and NumPy handle panel wrangling, statsmodels handles diagnostics and classical baselines, and PyTorch handles the sequence models.

Core libraries used

- pandas + pyarrow: read parquet data, keep only needed columns, sort series, and assemble summary tables.
- numpy: array operations, lag windows, seed control, and metric calculations.
- matplotlib + seaborn: EDA visuals, benchmark charts, forecast panels, and training curves.
- statsmodels: ADF/KPSS/STL/ACF-PACF diagnostics plus SES, Holt, and ARIMA baselines.
- torch: nn modules, TensorDataset/DataLoader pipelines, and CPU training for RNN, GRU, and LSTM models.

Adaptive splits for chosen series

Series	Train len	Val len	Train range	Validation range
Longest History	169	43	2826–2994	2995–3037
Highest Total Weight	124	32	2874–3000	3001–3032
Most Volatile	129	33	2747–2878	2879–2911
Most Stable	136	34	2874–3012	3013–3052

Where they show up

- 01_EDA.ipynb: schema checks, missingness, stationarity, STL, lag structure, and split sanity.
- 02_classical_codex.ipynb: representative-series baselines with SES, Holt, and ARIMA variants.
- 03_deep_sequence_models_chosen.ipynb: adaptive 80/20 splits, target-only windows, and recurrent training.

Adaptive Splits and Training Constraints

Takeaway Adaptive splits solve a feasibility problem, but they also change what the deep results can legitimately claim against the fixed-cutoff classical study.

Chronology preserved

Each adaptive split still keeps training strictly before validation. The change is in the cutoff location, not in the causal direction.

Why fixed cutoff fails

The highest-weight and most-stable series offer only seven training points under the global cutoff, which makes recurrent learning effectively impossible.

Training constraints

Training uses CPU and deterministic seeds; forecasts are recursive and open-loop, which is harder than teacher-forced one-step validation.

Interpretation rule

- The deep study is fair within its own setup.
- The final classical-vs-deep comparison must keep the split mismatch explicit.

Deep Model Design and Math

Takeaway The deep study uses simple recurrent architectures on lag windows; the real risk is not architectural complexity but whether recursive forecasting stays stable over the validation horizon.

Window input

Use the last L targets
as input context.

Predict the next value.

Architectures

RNN: simplest recurrent baseline

GRU: gated recurrence
with fewer parameters

LSTM: gated memory cell
for longer dependencies

Inference rule

Recursive forecast:
predict one step,
append it to the context,
and repeat.

Harder than one-step validation,
but closer to deployment.

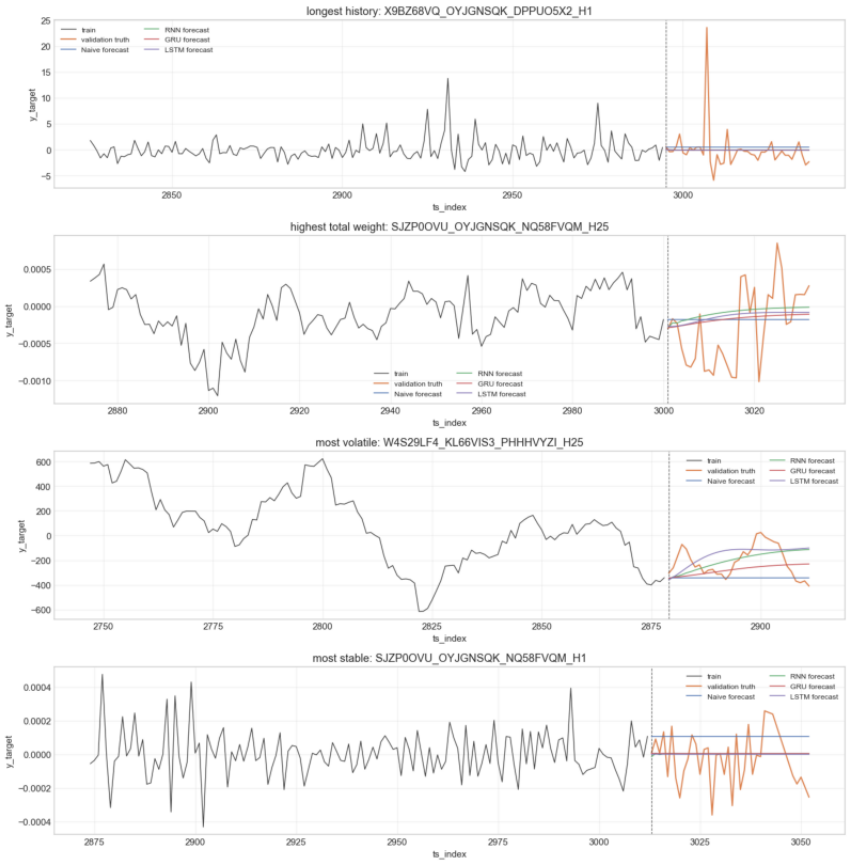
Evaluation setup

- Naive persistence is kept as a baseline on every chosen series.
- Training curves use teacher-forced validation for diagnostics, but the reported scored forecasts are recursive.
- This keeps the deep study transparent about what the loss curves do and do not prove.

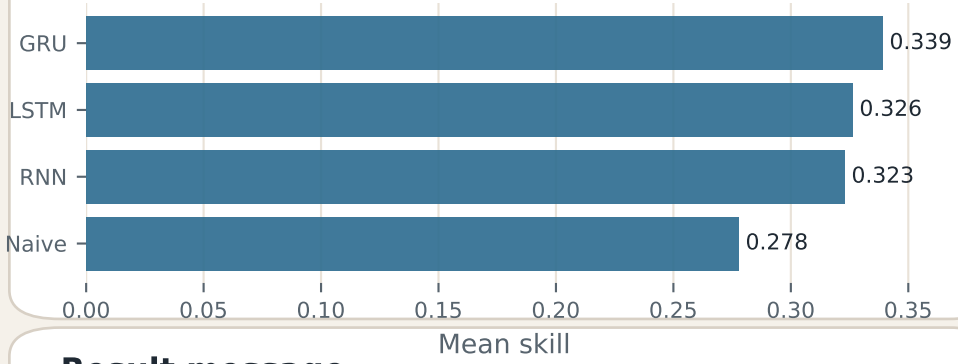
Deep Results

Takeaway Deep models help most on the highest-weight and longest-history cases, while the volatile series remains competitive with classical ARIMA and the most stable series is still hard to beat beyond a naive baseline.

Deep forecast plots



Deep leaderboard



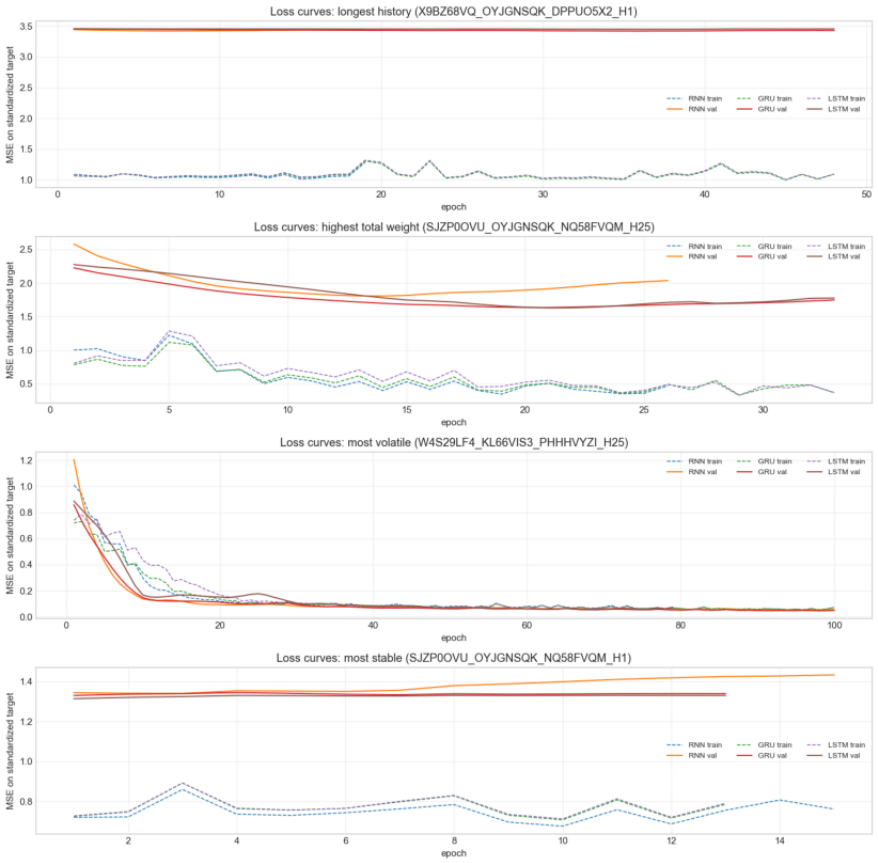
Result message

GRU leads on mean skill overall, LSTM edges the longest-history case, RNN stays best on the volatile case, and naive persistence still ties the most-stable series at zero skill.

Deep Training Curves

Takeaway The training curves are diagnostic rather than decisive: they show optimization behavior, but the scored forecasts still need to be judged on recursive validation performance.

Training curves



Per-series winners

Series	Best deep model	Skill
Highest Total Weight	GRU	0.511704
Longest History	LSTM	0.034827
Most Stable	Naive	0.000000
Most Volatile	RNN	0.819422

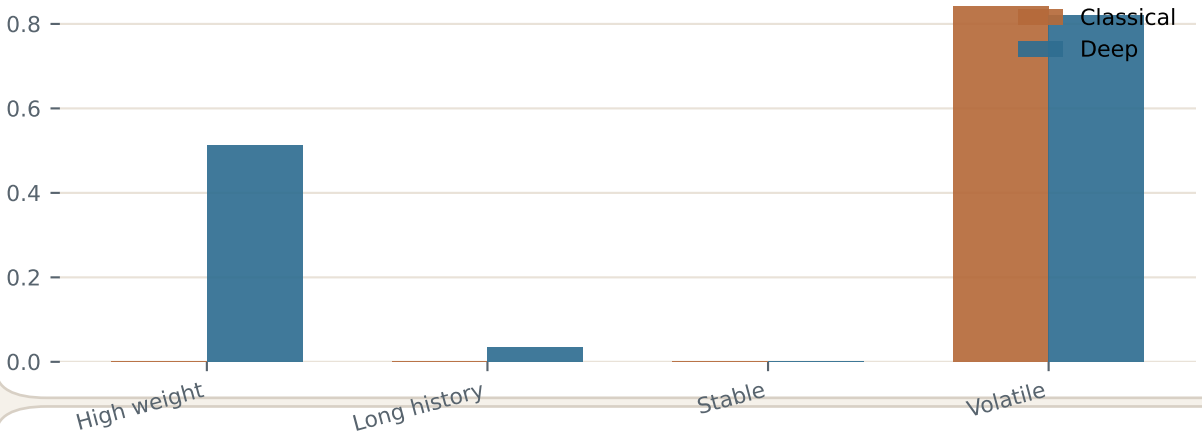
How to read this

- The curves help diagnose stability and overfitting trends.
- They do not replace recursive multi-step validation scores.
- The final comparison should still privilege forecast behavior over loss-curve aesthetics.

Final Comparison and Close

Takeaway The cleaned comparison is useful but not perfectly apples-to-apples: deep models gain from adaptive splits, while classical models stay on the harsher fixed-cutoff setup.

Best classical vs best deep



Final per-series comparison

Series	Best classical	Best deep
Highest Total Weight	0.0	0.511704
Longest History	0.0	0.034827
Most Stable	0.0	0.0
Most Volatile	0.8412	0.819422

Honest conclusion

- Deep models clearly help on the highest-weight case and slightly help on the longest-history case.
- Classical ARIMA remains highly competitive on the volatile series.
- The most-stable series remains difficult to improve beyond a naive forecast.

Limits and future work

- The split mismatch means the final ranking should be read as directional, not definitive.
- Future work should bring pooled feature-based ML and ensembles into the same cleaned evaluation pipeline.
- The main project lesson is that diagnostics and evaluation design matter as much as model family choice.